

Roteiro Completo de Apresentação - Compilador GYH (Parte do Felipe)

✂️ Preparação Antes de Gravar

1. Deixe abertos no seu editor: `GyhGrammar.g4`, `SymbolTable.java`, `Symbol.java`.
 2. Deixe preparados os testes: `programa_lexico.gyh`, `programa6.gyh` e `programa2.gyh`.
 3. Lembre-se de apontar o mouse para as linhas de código quando estiver falando sobre elas, pois a explicação será linha a linha.
-

🎬 Início da Gravação

[Ação] Abra a câmera/microfone e mostre a IDE com o arquivo `GyhGrammar.g4` aberto na **linha 209** (onde começam os tokens).

Felipe (Fala Exata): "Olá, professora. Eu sou o Felipe Rodrigues e juntamente com o Guilherme, nós desenvolvemos o nosso Compilador da linguagem GYH. Eu fiquei responsável por toda a parte da análise Léxica, análise Sintática e também pela análise Semântica do projeto, e é isso que eu vou demonstrar linha a linha agora."

1. Análise Léxica (Explicando Todos os Tokens)

[Ação] Vá rolando a tela lentamente da **linha 209** até a **242** conforme fala.

Felipe (Fala Exata): "Começando pelo Analisador Léxico, nós utilizamos o ANTLR para mapear todas as nossas expressões regulares. Do final do arquivo `GyhGrammar.g4`, na **linha 209** até a **220**, nós declaramos as palavras reservadas da linguagem, como `DEC`, `PROG`, `INT`, `REAL`, `LER`, `IMPRIMIR`, `SE`, `ENTAO`, `SENAO`, `ENQTO`, `INI` e `FIM`. Das **linhas 221 a 232**, definimos os nossos operadores aritméticos (+, -, *, /), os operadores relacionais (como <, <=, >, >=, ==, !=) e os lógicos (E, OU). Na **linha 237** nós definimos o token `Var`, que aceita uma letra minúscula seguida de letras ou números, formando nossos identificadores. Na **linha 238** o `NumInt` aceita um ou mais dígitos, e na **239** o `NumReal` aceita dígitos com ponto flutuante. A `Cadeia` na **240** aceita qualquer caractere entre aspas duplas. Por fim, nas **linhas 241 e 242**, o token `Comment` e o `WS` (espaços em branco) usam a diretiva `-> skip`. Isso garante que o analisador léxico descarte comentários e espaços automaticamente, enviando para o sintático apenas os tokens que realmente importam."

2. Análise Sintática (Explicando as Regras)

[Ação] Suba o arquivo `GyhGrammar.g4` para a **linha 64** (onde começa a regra `programa`). Vá rolando conforme explica.

Felipe (Fala Exata): "Subindo para a **linha 64**, temos o nosso Analisador Sintático. A regra raiz `programa` dita a estrutura obrigatória: o código deve começar com um delimitador `:`, a palavra `DEC`, depois a nossa regra de `listaDeclaracoes`, seguida por `: PROG` e a `listaComandos`. Na **linha 73**, a `listaDeclaracoes` utiliza recursão para permitir a declaração de múltiplas variáveis. Pulando para a **linha 82**, a `listaComandos` também é

recursiva, e a regra comando na linha 85 ramifica para todas as possibilidades da linguagem: comandos de atribuição, entrada, saída, condição (if), repetição (while) e subalgoritmos (blocos de código). Um detalhe importante das nossas expressões matemáticas, a partir da linha 98: nós separamos em `expressaoAritmetica`, `termoAritmetico` e `fatorAritmetico`. Nós fizemos isso para garantir a precedência sintática, onde multiplicações e divisões são resolvidas antes de somas e subtrações. Como o ANTLR v4 lida com recursão à esquerda naturalmente, as expressões ficaram bem limpas e diretas. O mesmo fizemos para as expressões relacionais a partir da linha 116."

3. Análise Semântica (Explicando Todos os Métodos Java)

[Ação] Troque a aba para os arquivos `SymbolTable.java` e `Symbol.java` rapidamente. Depois, volte para o `GyhGrammar.g4` na **linha 16** (bloco `@members`).

Felipe (Fala Exata): "Para o Analisador Semântico, eu implementei as validações utilizando código Java puro acoplado na gramática. Eu criei a classe `Symbol`, que guarda o nome, o tipo e o valor de uma variável. A classe `SymbolTable` armazena esses símbolos dentro de um `HashMap` para busca rápida. Voltando para o `GyhGrammar.g4`, na linha 16, no bloco `@members`, nós instanciamos essa `SymbolTable` globalmente. E aqui eu implementei os 4 principais métodos semânticos do projeto:

1. Na linha 29, o `verificaEDeclaraVariavel`: chamado sempre que uma variável é declarada no bloco `DEC`. Ele verifica se a variável já existe no `HashMap`; se não existir, ele adiciona. Se já existir, ele acusa o Erro Semântico de 'Variável já declarada'.
2. Na linha 39, o `verificaUsoVariavel`: acionado antes de qualquer leitura, escrita ou atribuição. Ele verifica se a variável está no `HashMap`. Se não estiver, lança o erro de 'Variável não declarada'.
3. Na linha 45, o `verificaTipoOpAritmetica`: ele pega os tipos de dois operandos matemáticos. Se algum deles for `REAL`, ele promove o resultado para `REAL`, caso contrário, mantém como `INT`.
4. E na linha 53, o `verificaTipoAtribuicao`: que impede, por exemplo, que uma variável declarada como `INT` receba o resultado de uma expressão `REAL`, acusando 'Incompatibilidade de tipos'. Nós chamamos essas funções injetadas diretamente nas regras sintáticas, como na linha 76 (onde chamo a declaração) e na linha 89 (onde chamo a verificação de uso da atribuição)."

4. Demonstração dos 3 Erros no Terminal

[Ação] Abra o terminal da sua IDE para rodar os testes. Abra a aba com o arquivo `programa_lexico.gyh`.

Felipe (Fala Exata): "Agora eu vou provar o funcionamento prático mostrando as barreiras de erro. Primeiro, o Erro Léxico: no arquivo `programa_lexico.gyh`, coloquei um caractere de arroba (@) na atribuição, que não pertence a nenhum token. Vou compilar..." **[Ação]** Rode o comando para compilar o arquivo. **Felipe (Fala Exata):** "Como podem ver, o lexer captura imediatamente com *token recognition error at: '@'.*"

[Ação] Troque para a aba do `programa6.gyh` e mostre a linha 10. **Felipe (Fala Exata):** "Agora o Erro Sintático: no `programa6.gyh`, linha 10, tentaram fazer uma atribuição usando o duplo igual (==), que é um operador relacional, e não de atribuição (: =). Vou rodar..." **[Ação]** Rode o comando para compilar o arquivo. **Felipe (Fala Exata):** "O analisador sintático barra na hora com *no viable alternative at input 'fatorial = ='*, pois a sequência violou a regra."

[Ação] Troque para a aba do `programa2.gyh` e aponte para a linha 15 (`num4`). **Felipe (Fala Exata):** "Por último, o Erro Semântico: no `programa2.gyh`, linha 15, o SE tenta consultar a variável `num4`, mas ela nunca foi declarada na sessão DEC. Vou rodar..." **[Ação]** Rode o comando para compilar o arquivo. **Felipe (Fala Exata):** "A mensagem programada pela minha classe Java aparece no console: *Erro Semântico: Variável não declarada! -> num4*. A nossa tabela de símbolos evitou que o compilador avançasse com um estado inválido."

[Ação] Conclusão da sua parte. **Felipe (Fala Exata):** "Com isso, demonstro que as 3 análises — Léxica, Sintática e Semântica — operam linha por linha, cobrindo todo o escopo da linguagem GYH. Agora passo a palavra para o Guilherme, que detalhará os comandos de Geração de Código e rodará a compilação final para C. Obrigado!"

(Fim da sua parte no vídeo!)